

ECEN 428/722 - Post Lab Report

Lab Number: 1

Lab Title: CORDIC Circuit Design using Vitis HLS

Section Number: 602

Student's Name: Sai Vikhyat Pattar

Student's UIN: 936001854

Date: 9/18/2025

TA: Cristhian Roman Vicharra

Objective:

This lab focused on the implementation of the **CORDIC (Coordinate Rotation Digital Computer)** algorithm and its practical applications. CORDIC is an efficient iterative method for computing trigonometric, hyperbolic, and other mathematical functions, as well as performing division and multiplication, without the use of complex multipliers. Instead, it relies on simple shift and add operations to rotate vectors in circular, linear, or hyperbolic coordinate systems. By performing a sequence of small, predefined rotations, the algorithm converges to the desired value. Its hardware-friendly nature makes it particularly suitable for applications such as digital signal processing, robotics, and scientific calculators where resource-efficient, high-speed computation is required.

To calculate sine and cosine for a given input angle, the initial conditions are set as $x_0 = 1/G$, $y_0 = 0$, and $z_0 = \text{input angle}$. After a fixed number of iterations, the values converge such that $X_k \approx \cos(z_0)$ and $Y_k \approx \sin(z_0)$, thereby producing the sine and cosine results of the input.

$$x_k = G(x_0 \cos z_0 - y_0 \sin z_0)$$

$$y_k = G(x_0 \sin z_0 + y_0 \cos z_0)$$

$$z_k = z_0 - 2^{-k} z_0$$

As X_0 is $1/G$ and Y_0 is 0, we get X_k as $\cos(z_0)$ and Y_k as $\sin(z_0)$ thus calculating the sine and cosine for the given angle.

Design:

1. Download and extract the provided lab code.
2. Open Vitis HLS IDE, create a new project, and add the source files (Basic_cordic.cpp, Basic_cordic.h, Basic_cordic_tb.cpp, hyperbolic_cordic.cpp, hyperbolic_cordic.h, hyperbolic_cordic_tb.cpp).
3. Select the target board xc7z020clg400-1.
4. Resolve any syntax errors, then run C Simulation for the basic_arctan and hyperbolic_cordic implementations. The testbench checks results against reference values, reporting an error if the deviation exceeds 1%.
5. Run C Synthesis to generate the RTL design from the C code.
6. Perform RTL/C Co-Simulation, reusing the C testbench to verify the generated RTL.
7. Confirm correctness by analyzing the simulation waveforms.

Results:

```
Vitis Messages  cordic_lab1::c-simulation X
54 Input sine: 0.865967
55 Input cosine: 0.5
56 Reference arctan(x): 1.04712
57 Your arctan(x): 1.04688
58 Input sine: 0.642578
59 Input cosine: 0.766113
60 Reference arctan(x): 0.697754
61 Your arctan(x): 0.698242
62 Input sine: 0.258545
63 Input cosine: 0.96582
64 Reference arctan(x): 0.26123
65 Your arctan(x): 0.260742
66 Input sine: -0.342285
67 Input cosine: 0.939453
68 Reference arctan(x): -0.349365
69 Your arctan(x): -0.350098
70 Input sine: -0.500244
71 Input cosine: 0.865967
72 Reference arctan(x): -0.523926
73 Your arctan(x): -0.523438
74 Input sine: -0.819336
75 Input cosine: 0.573486
76 Reference arctan(x): -0.960205
77 Your arctan(x): -0.959961
78 Input sine: -0.939941
79 Input cosine: 0.341797
80 Reference arctan(x): -1.22217
81 Your arctan(x): -1.22266
82 Input sine: -0.999512
83 Input cosine: 0.0344238
84 Reference arctan(x): -1.53638
85 Your arctan(x): -1.53564
86 [Success] Your implementation of arctan is correct!
87 INFO: [SIM 211-1] CSim done with 0 errors.
88 INFO: [SIM 211-3] ***** CSIM finish *****
89 INFO: [HLS 200-112] Total CPU user time: 4.05 seconds. Total CPU system time: 0.71 seconds. Total elapsed time: 4.83 seconds; peak allocated memory: 482.996 MB.
90 INFO: [vitis-run 60-791] Total elapsed time: 0h 0m 8s
91 C-simulation finished successfully
92
```

Fig 1: C simulation result of cordic arctan

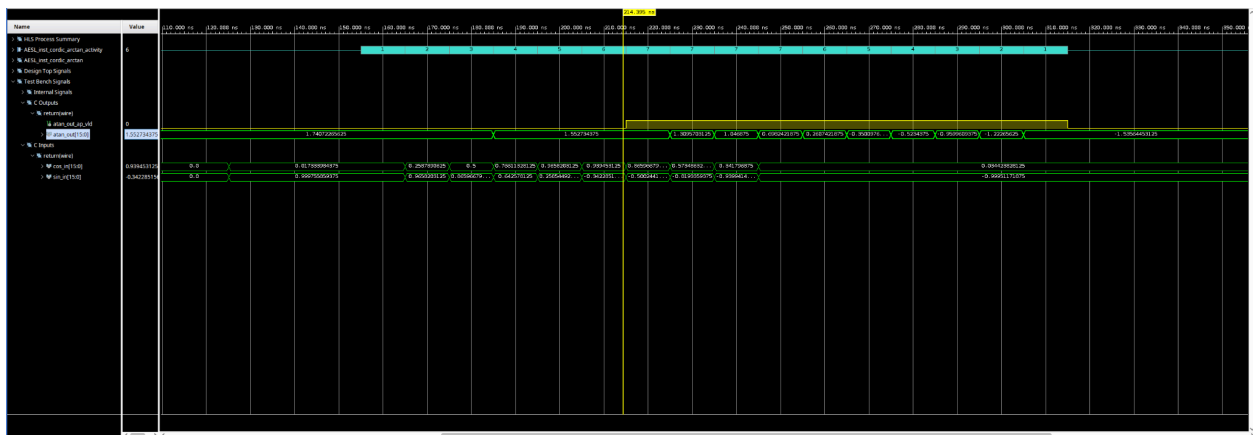


Fig 2 Behavioral simulation waveform of Cordic arctan.

Explanation:

For determining the correctness of the output for cordic arctan, we initially send out the sine and cosine values for that corresponding angle. We then set the initial conditions for the cordic to work in vectoring mode so as to find the tan inverse of the input values. We see this output in the atan_out variable, where we get the values in radians (a 12 point fixed real number).

```
45 [Success] Your implementation of hyperbolic sine and cosine are correct!
46 INFO: [SIM 211-1] CSim done with 0 errors.
47 INFO: [SIM 211-3] ***** CSIM finish *****
48 INFO: [HLS 200-112] Total CPU user time: 3.91 seconds. Total CPU system time: 0.8 seconds. Total elapsed time: 4.92 seconds; peak allocated memory: 482.965 MB.
49 INFO: [vitis-run 60-791] Total elapsed time: 0h 0m 8s
50 C-simulation finished successfully
51
```

Fig 3: C simulation result of cordic in the hyperbolic co-ordinate system

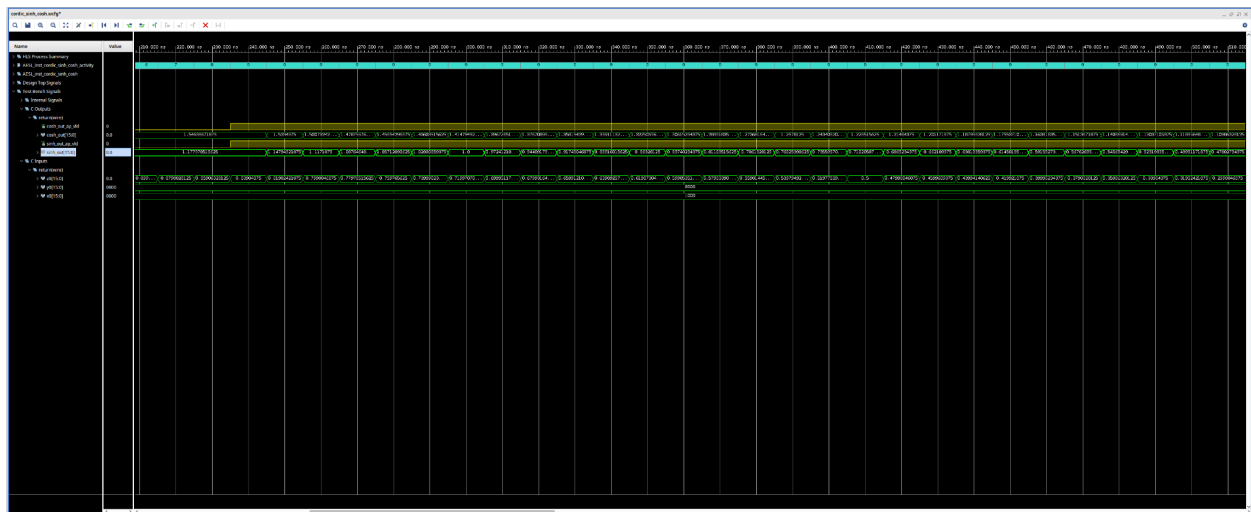


Fig 4: Waveform for hyperbolic cordic

Explanation:

CORDIC's versatility allows it to operate in both circular and hyperbolic coordinate systems. By making minor adjustments—such as initializing variables differently—the algorithm can be run in **rotation mode** for the hyperbolic system. Using the same initial setup as the trigonometric case, the outputs generated are **$\sinh(z_0)$** and **$\cosh(z_0)$** .

Conclusion:

This lab demonstrated the efficiency and adaptability of the **CORDIC algorithm**, first introduced by Volder in 1959. Traditionally, functions such as trigonometric, exponential, multiplication, and division were computationally intensive. CORDIC reduces these to a series of simple additions and bit shifts, making it highly suitable for digital hardware. The number of iterations provides a trade-off between precision and speed, offering flexibility across applications. By adjusting initial conditions, CORDIC can compute sine, cosine, and arctan, and with slight modifications, extend to hyperbolic functions. Overall, CORDIC proves to be a powerful and resource-efficient method for implementing complex mathematical operations in digital circuits.